

6: Inter-Process Communication

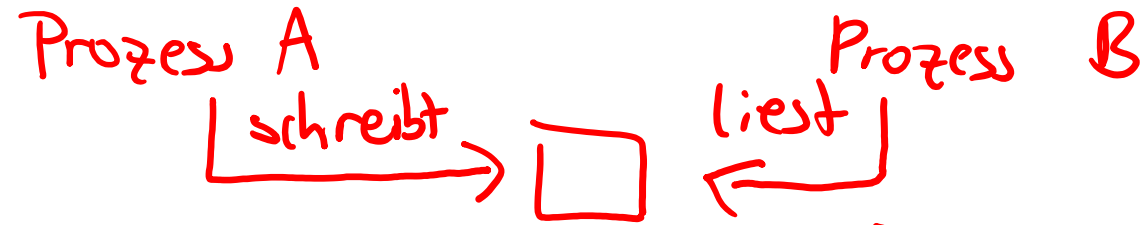
TUTORIUM 24 - BuS 2024

Sciebo-Ordner für alle, die keinen Zugriff auf unsere Shared folder haben



Wiederholung: Möglichkeiten zur Interprozesskommunikation

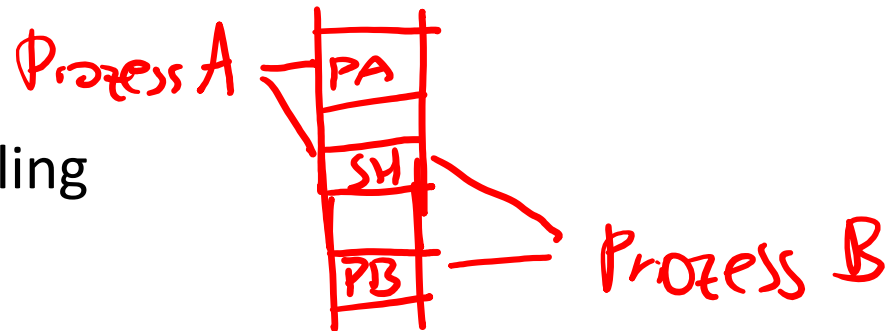
- Pipes



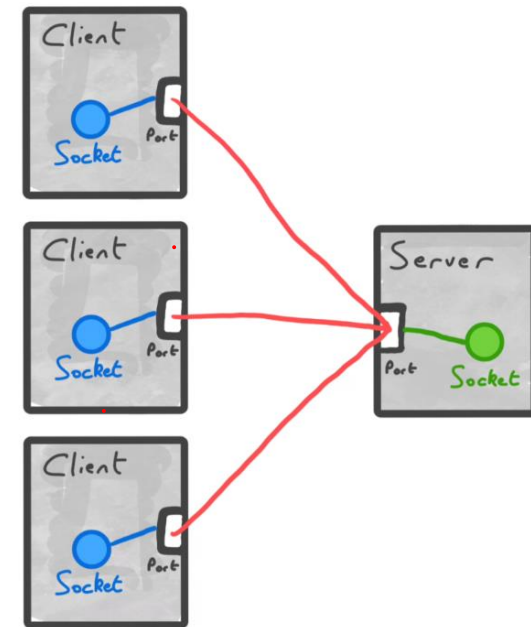
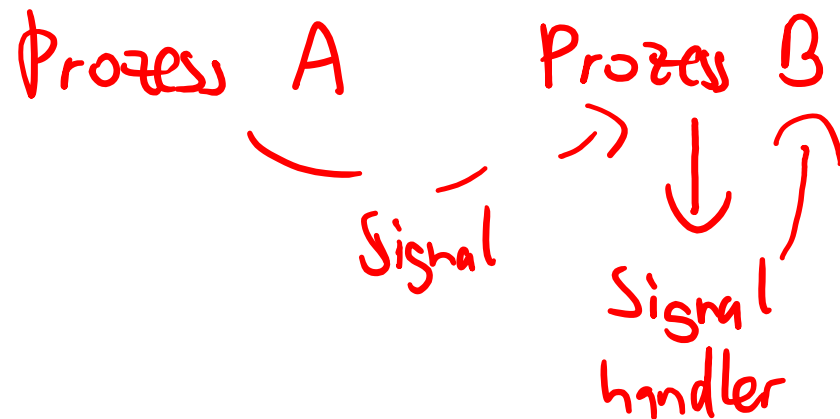
- Shared Memory



- Signaling

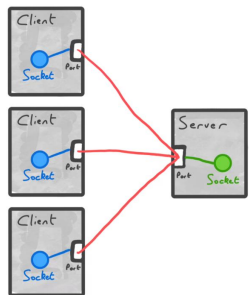


- Sockets



Wiederholung: Möglichkeiten zur Interprozesskommunikation

- Pipes
 - Unidirektional – Verbinden Filedesktoren so, dass ein Prozess in die Pipe reinschreibt und der andere ausliest
- Shared Memory
 - Beliebig viele Prozesse können in gemeinsamen Speicherbereich lesen und schreiben – benötigt Synchronisation
- Signaling
 - Asynchrone Benachrichtigung von einem Prozess an den nächsten
 - Bei Ankunft des Signals wird vom Empfänger ein Signal handler ausgeführt, anschließend das Programm weiter ausgeführt – „Software interrupt“
- Sockets
 - Besondere Files, die an eine Netzwerkinterface angebunden sind
 - Stellen uns ein Interface zur Verfügung, indem wir über Netzwerkkommunikation Nachrichten senden können
 - Vielseitig einsetzbar, gut erweiterbar, aber auch sehr komplex



Frage 1:

Pipes vs. Shared Memory

Pipes und Shared Memory sind zwei Möglichkeiten, um Informationen zwischen zwei Prozessen auszutauschen. Diskutiert die **Vor- und Nachteile** beider Varianten.

Frage 1: Pipes vs. Shared Memory – Vor- und Nachteile

- Pipes: *einfach*
 - **Vorteil:** bei richtiger Nutzung keine Synchronisation nötig
 - **Nachteile:**
 - Mehraufwand bei der Kommunikation: *viel Overhead*
read() und write() sind syscalls und verursachen Kontextwechsel
 - Speicher muss zwischen Prozessen kopiert werden
- Shared Memory
 - **Vorteil:** Speicher muss nicht kopiert werden, da die Prozesse sich denselben Bereich teilen *schneller*
 - **Nachteil:** Synchronisationsmechanismen sind erforderlich, um Race Conditions oder sonstige Inkonsistenzen zu verhindern
komplex

Frage 2:

Named Pipes vs. Files

In der Vorlesung wurde gesagt, dass Named Pipes sehr ähnlich zu normalen Dateien sind.

Was sind die **wichtigsten Unterschiede** zwischen den beiden?

Frage 2: Named Pipes vs. Files: Differences

- Named Pipes speichern keine Daten in Dateien:
Sobald die Daten einmal aus der Pipe gelesen sind, „verschwinden“ sie und können **nicht** noch einmal gelesen werden
- Suchen wie bei normalen Dateien ist nicht möglich
- Serieller Zugriff, daher auch die Bezeichnung FIFO, siehe command `mkfifo`
- Normale Dateien bieten im Kontrast dazu wahlfreien Zugriff

Frage 3:

Web Server Configuration

Typischerweise lesen Webserver eine Konfigurationsdatei ein und wenden darauf basierend dann Regeln an. Um zu verhindern, dass wir den Server bei jeder Regeländerung neustarten müssen, wollen wir ein Feature implementieren, dass uns eine solche Regeländerung ohne Neustart erlaubt.

Diskutiert mögliche Vor- und Nachteile der folgenden Methoden:

- Signaling ✓
- Shared Memory ✗
- Pipes
- Sockets

Frage 3: Web Server Configuration

- Signals ✓
 - Einfach zu implementieren und zu nutzen
 - Kein neuer Maincode nötig, nur der kill-command muss zum Hauptprogramm gesendet werden
 - Nur der Signal Handler muss geschrieben werden
- Shared Memory ✗
 - Komplett neues Programm mit Implementierung von Shared Memory
 - Hauptprogramm muss synchronisiert werden
 - Es müssen nur wenig Daten übertragen werden
- Pipes
- Sockets

Frage 3: Web Server Configuration

- **Signals**
- *Shared Memory*
- Pipes
 - Vorteil gegenüber Shared Memory wie in Frage 1: keine Synchronisation und einfacher zu implementieren
 - Performanceaufwendige Lösung, Pipe muss regelmäßig gecheckt werden; viel Overhead
- Sockets
 - Für Netzwerkkommunikation sinnvoll
 - Kommunikation innerhalb eines Gerätes ist unnötig komplex im Vergleich zu Pipes oder Shared Memory

Frage 4:

Autonomous Vehicle Control System

Ein Kontrollsystem für autonomes Fahren hat viele komplexe Subsysteme, die in Echtzeit Daten von z.B. Sensoren lesen müssen, Entscheidungen treffen und Befehle behandeln müssen. Daher braucht geringe Latenzen, um schnelle Anpassungen an dynamische Verkehrssituationen vornehmen zu können. Weiterhin notwendig sind eine hohe Bandbreite, um die Vielzahl an Sensoreneingangsdaten zu handeln und effektive Synchronisationsmechanismen, um zeitkritische Entscheidungen zu ermöglichen.

Diskutiert mögliche Vor- und Nachteile der folgenden Methoden:

- Signaling
- Shared Memory
- Pipes
- Sockets

Frage 4: Autonomous Vehicle Control System

- Signals
 - Effizient für Benachrichtungen oder Trigger
 - Einfach zu implementieren, daher gut geeignet für Benachrichtigungen über Events zwischen Subsystemen
 - Nicht geeignet für komplexe Datentransporte
 - Hohe Latenzen möglich, da bei Signaling keine Zeitgarantie besteht
- Shared Memory
 - Daten können direkt, mit wenig Latenz und mit hohem Durchsatz auch in zeitkritischen Umgebungen übertragen werden
 - Sehr gute Synchronisation ist hier entscheidend
 - Umgang kann schwierig sein, wird aber auch durch gute Performance belohnt

Frage 4: Autonomous Vehicle Control System

- Pipes
 - Einfach zu nutzen, gut um Daten Schritt-für-Schritt zu übertragen
 - Große Mengen an Daten können so nicht gut übertragen werden, s. Overhead, serielle Übertragung, one-way, etc.
- Sockets
 - Können Flexibilität ermöglichen, indem sie Netzwerkkommunikation ermöglichen
 - Verlässlich und skalierbar
 - Nachrichten werden langsamer versendet, da sie durchs Netzwerk versendet werden, erneutes Set-up dauert länger